

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

ОТЧЕТ
по лабораторной работе №4
по дисциплине «Построение и анализ алгоритмов»
Тема: Поиск подстроки в строке

Студент гр. 4343

Васильев А. С.

Преподаватель

Жангиров Т. Р.

Санкт-Петербург

2026

Задание

Задача 1.

Реализуйте алгоритм КМП и с его помощью для заданных шаблона P ($|P| \leq 25000$) и текста T ($|T| \leq 5000000$) найдите все вхождения P в T .

Вход:

- Первая строка — P
- Вторая строка — T

Выход:

индексы начал вхождений P в T , разделённые запятой; если P не входит в T , то вывести -1.

Задача 2.

Заданы две строки A ($|A| \leq 5000000$) и B ($|B| \leq 5000000$).

Определить, является ли A циклическим сдвигом B (это значит, что A и B имеют одинаковую длину и A состоит из суффикса B , склеенного с префиксом B).

Например, defabc является циклическим сдвигом abcdef.

Вход:

- Первая строка — A
- Вторая строка — B

Выход:

Если A является циклическим сдвигом B , то индекс начала строки B в A ; иначе вывести -1. Если возможно несколько сдвигов, вывести первый индекс.

Теоретические сведения

Алгоритм Кнута — Морриса — Пратта предназначен для поиска шаблона в тексте за линейное время.

Основная идея алгоритма заключается в том, что при несовпадении символов не нужно начинать сравнение шаблона с начала. Вместо этого используется заранее вычисленная префикс-функция, которая показывает, на какую позицию можно откатиться внутри шаблона.

Префикс-функция $pi[i]$ для строки S — это длина наибольшего собственного префикса строки $S[0:i+1]$, который одновременно является её суффиксом.

Описание алгоритмов

Задача 1.

Для поиска всех вхождений шаблона P в текст T используется алгоритм Кнута — Морриса — Пратта.

Сначала для шаблона P строится префикс-функция pi . Значение $pi[i]$ показывает длину наибольшего собственного префикса подстроки $P[0:i+1]$, который одновременно является её суффиксом. Эта информация нужна для того, чтобы при несовпадении не начинать сравнение шаблона заново, а выполнить откат к уже известной совпавшей части.

После построения префикс-функции выполняется проход по тексту T . Переменная j хранит количество символов шаблона, которые уже совпали. Если текущие символы текста и шаблона совпадают, j увеличивается. Если происходит несовпадение, j откатывается по префикс-функции $j = pi[j - 1]$.

Когда j становится равным длине шаблона, найдено полное вхождение. Индекс начала вхождения вычисляется так: $start = i - |P| + 1$.

Сложность алгоритма:

Пусть $m = |P|$ — длина шаблона, $n = |T|$ — длина текста.

Построение префикс-функции занимает $O(m)$, так как она вычисляется только для шаблона P .

Поиск всех вхождений шаблона P в тексте T занимает $O(n)$, так как алгоритм выполняет один проход по тексту без возврата назад.

Итоговая временная сложность алгоритма составляет $O(n + m)$.

Дополнительная память составляет $O(m + k)$, где m — длина шаблона, а k — количество найденных вхождений.

Задача 2.

Для проверки, является ли строка A циклическим сдвигом строки B , используется алгоритм Кнута — Морриса — Пратта.

Сначала проверяется, равны ли длины строк A и B . Если длины различаются, то строка A не может быть циклическим сдвигом строки B , поэтому сразу выводится -1.

Если длины строк равны, задача сводится к поиску строки B в строке $A + A$. Это связано с тем, что любой циклический сдвиг строки обязательно содержится внутри строки, полученной склейкой этой строки с самой собой. Например, если $A = \text{defabc}$, то $A + A = \text{defabcdefabc}$, и строка $B = \text{abcdef}$ начинается в ней с индекса 3.

Для поиска строки B в $A + A$ используется алгоритм КМП. Сначала для строки B строится префикс-функция pi . Значение $pi[i]$ показывает длину наибольшего собственного префикса подстроки $B[0:i+1]$, который одновременно является её суффиксом. Эта информация используется для отката при несовпадении символов.

После построения префикс-функции выполняется поиск строки B в виртуальной строке $A + A$. При этом строка $A + A$ явно не создаётся: сначала выполняется проход по строке A , затем по началу строки A . Переменная j хранит количество уже совпавших символов строки B . Если символы совпадают, j увеличивается. Если происходит несовпадение, выполняется откат по префикс-функции: $j = pi[j - 1]$.

Когда j становится равным длине строки B , найдено полное вхождение. Индекс начала этого вхождения является ответом задачи. Если вхождение не найдено, выводится -1.

Сложность алгоритма:

$n = |A| = |B|$ — длина строк.

Построение префикс-функции для строки B занимает $O(n)$.

Поиск строки B в виртуальной строке $A + A$ занимает $O(n)$, так как достаточно проверить не более $2n - 1$ символов.

Итоговая временная сложность алгоритма составляет $O(n)$.

Дополнительная память составляет $O(n)$, так как хранится префикс-функция для строки B .

Тестирование

task1.py:

Входные данные	Вывод
ab abab	0,2
aa aaaa	0,1,2
abc defdef	-1
a aaaa	0,1,2,3

task2.py:

Входные данные	Вывод
defabc abcdef	3
abcdef abcdef	0
ab abab	-1
abc acb	-1